Project Report: AI Career Advisor

1. Problem Statement

Many students face "decision paralysis" when choosing a career path because they cannot easily map their natural interests (like logic, design, or biology) to specific professional outcomes. Traditional career counselling is often inaccessible or limited to a few popular fields. There is a need for a **rational agent** that can store vast professional knowledge and provide unbiased, logic-based recommendations.

2. Why It Matters

A person's career choice is one of the most impactful decisions in their life. By building an **Expert System**, we can provide a scalable tool that helps users navigate complex professional landscapes using objective facts rather than guesswork. This project demonstrates how AI can serve as a supportive tool in human decision-making.

3. Approach & Methodology

I chose a **Hybrid AI Architecture** to solve this problem, combining two distinct programming paradigms:

- **Logic Engine (Prolog):** I used Prolog for the "Knowledge Representation" because it is designed for symbolic reasoning. Unlike standard code, Prolog allows me to define **facts** (e.g., what a doctor needs) and **rules** (how to match a user to a career) independently.
- **User Interface (Python):** I used Python to build the "Intelligent Agent" that interacts with the user. Python handles the input/output and uses the pyswip library to "consult" the Prolog brain.

4. Key Decisions

- **Direct Inference Model**: I decided to focus on a direct mapping between one interest and one career. This ensures the AI provides immediate, clear feedback by matching a specific user aptitude to a professional field using Prolog's built-in Unification process.
- **Dynamic Database:** I decided to use the assertz command in Python. This allows the AI to "learn" the user's interests during the session and add them to its internal memory without permanently changing the source code.

```
# Insert the user's interest as a fact into the Prolog engine.
prolog.assertz(f"likes(user, {trait})")
```

5. Implementation & Results

The system successfully processes user input and returns logical career matches from various domains, including Engineering, Medicine, Law, and the Arts.

```
============== RESTART: C:\Users\react\Desktop\aimlproject\main.py =============
 AI Universal Career Advisor is UP and RUNNING!
Available Options: math, python, biology, empathy, logic, oratory, creativity, design, physics, networking, chemistry, psychology, finance

What do you like doing?: math

Based on your interest, here are some career paths you might enjoy:
-> You might enjoy being a Data Scientist
-> You might enjoy being a Chartered Accountant
-> You might enjoy being a Research Scientist
```

Result when the trait is in the knowledge base

```
================================================================ RESTART: C:\Users\react\Desktop\aimlproject\main.py ================================================================
 AI Universal Career Advisor is UP and RUNNING!
Available Options: math, python, biology, empathy, logic, oratory, creativity, design, physics, networking, chemistry, psychology, finance

What do you like doing?: dancing

That's above my current knowledge. You're on your own, Buddy!
```

Result when the trait is not in the domain

```
================================================================ RESTART: C:\Users\react\Desktop\aimlproject\main.py ================================================================
Error: Check if SWI-Prolog is installed and data.pl is in this folder.
```

Error Handling: When the Knowledge base is missing

6. Challenges Faced

- **Environment Integration:** A significant challenge was connecting the Python environment with the SWI-Prolog dynamic link library. I solved this by manually configuring the System PATH variables so Python could "find" the Prolog engine.
- **Logic Mapping:** Translating subjective human careers into strict logical predicates required careful thought to ensure the rules were neither too broad nor too narrow.

7. What I Learnt

Through this project, I gained a deep understanding of:

- **Knowledge Representation:** How to structure real-world information into a format an AI can reason with.
- **Hybrid Systems:** The power of combining different programming languages to utilize the strengths of each (Python for flexibility, Prolog for logic).
- **Agent Design:** How to create a functional software agent that perceives user input and acts rationally to provide a solution